

05-analytics-engineering

15. Analytics Engineering Architecture

Helios treats analytics as software. Every metric the Diagnose and Decompose agents consume is the output of a deterministic, version-controlled, tested transformation pipeline — never an ad-hoc query an LLM authored. This section specifies the layered architecture, the ELT flow, idempotency and incremental strategy, the testing pyramid, freshness SLAs, CI/CD, environments, code review, and lineage.

15.1 Layered architecture (sources -> staging -> intermediate -> marts -> semantic)

The pipeline is a strict five-layer DAG. Data flows in one direction; no layer reaches across or upward.

```
bigquery-public-data.ga4_obfuscated_sample_ecommerce
  events_YYYYMMDD (date-sharded, rows = events)
    |
[SOURCES]   src_ga4 (declared in sources.yml; no SQL, contract only)
    |
[STAGING]   stg_ga4__events      stg_ga4__event_params
            (1:1 with source, renamed/typed, light cleaning, views)
    |
[INTERMEDIATE] int_ga4__sessionized int_ga4__funnel_steps
            (business logic: session keys, channel grouping, funnel flags)
    |
[MARTS] core: fct_sessions fct_funnel fct_daily_funnel
            dim_users dim_items dim_channels dim_date
            finance: fct_orders fct_order_items
            growth: (rollups feeding stats-mcp)
    |
[SEMANTIC]   models/semantic (governed metric defs; the ONLY SQL path
            for the LLM, surfaced through semantic-mcp.get_metric /
            build_query)
```

- **Sources** (`src_ga4`) declare the upstream `events_*` shards as a contract. No transformation; they pin schema, freshness, and partition expectations so the build fails loudly if Google changes the export.
- **Staging** (`stg_ga4__*`) is 1:1 with the source, materialized as **views**. Responsibilities: rename to snake_case, cast types, flatten nothing structural yet, and expose the de facto keys (`user_pseudo_id` , `event_timestamp`). `stg_ga4__event_params` unnests the `event_params` ARRAY into a long key/value table so downstream models never re-implement UNNEST.
- **Intermediate** (`int_ga4__*`) holds reusable business logic that more than one mart needs: sessionization (the (`user_pseudo_id` , `ga_session_id`) key, landing page, session-scoped source/medium, channel group) and funnel-step flags. Materialized as ephemeral or table depending on cost.
- **Marts** are the consumption layer, split into `core` , `finance` , `growth` . Facts are grain-explicit (`fct_sessions` = one row per session; `fct_orders` = one row per `transaction_id`). Dims are conformed.
- **Semantic** is the governance boundary. Every canonical metric (`session_conversion_rate` , `revenue` , `aov` , `revenue_per_session` , ...) is defined exactly once here. `semantic-mcp` reads these definitions; the

LLM composes them but never writes the SQL.

15.2 ELT flow

Extract-Load is free: the GA4 export already lives in BigQuery as `bigquery-public-data`. Helios is therefore an **ELT** system where the "T" is the entire dbt project. A run is:

1. **Scheduler trigger** (Cloud Scheduler / cron) kicks the Orchestrator on the autonomous cadence.
2. `dbt build --select state:modified+` (CI) or `dbt build` (prod refresh) compiles, runs, and tests the DAG against the prod dataset.
3. Marts and the semantic layer materialize.
4. `warehouse-mcp.reconcile(metric, grain)` is invoked to assert marts agree with canonical raw totals.
5. The agent pipeline (Monitor -> Decompose -> Diagnose -> Prescribe -> Narrator, with the Critic gating) runs against the semantic layer only.

15.3 Idempotency and incremental vs full-refresh

Every model is **idempotent**: re-running over the same `event_date` partitions yields byte-identical output. This is guaranteed by partition-by-date plus `insert_overwrite`.

- **Full refresh** is the default for the small public dataset (~3 months, Nov 2020–Jan 2021). `dbt build --full-refresh` rebuilds everything cheaply and is the safe reset.
- **Incremental** is the production pattern, partitioned by `event_date` (`DATE`). New shards (e.g. a freshly landed `events_YYYYMMDD`) are processed with `incremental_strategy='insert_overwrite'` , which atomically replaces only the touched partitions. A late-arriving or corrected shard reprocesses cleanly because the whole partition is overwritten — no dedup logic, no drift.

```
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    cluster_by=['device_category', 'channel_group']
) }}
```

The incremental predicate prunes the source to the lookback window:

```
{% if is_incremental() %}
    where event_date >= date_sub(_dbt_max_partition, interval 3 day)
{% endif %}
```

A 3-day lookback absorbs GA4's late event landing without reprocessing the full history.

15.4 Testing strategy (the pyramid)

Layer	Test type	Tooling	What it proves
Schema	<code>unique</code> , <code>not_null</code> , <code>accepted_values</code> , <code>relationships</code>	dbt generic tests	structural contract holds

Layer	Test type	Tooling	What it proves
Data	freshness, row-count thresholds, value ranges	dbt + <code>dbt_utils</code>	data is sane, not just well-typed
Unit	seeded input -> asserted output for a transform	dbt <code>unit_tests</code> (<code>given / expect</code>)	sessionization & channel logic are correct
Reconciliation	mart total == raw canonical total	custom singular test + <code>warehouse-mcp.reconcile</code>	marts didn't silently drift

- **Schema tests** live in `schema.yml` beside every model. Examples: `fct_orders.transaction_id` is `unique + not_null`; `dim_channels.channel_group` is `accepted_values` of the ten canonical channels; `fct_funnel.user_pseudo_id` has a `relationships` test to `dim_users`.
- **Data tests** assert `session_conversion_rate` between 0 and 1, `revenue ≥ 0`, and source freshness (latest `event_date` within SLA).
- **Unit tests** seed a handful of synthetic events and assert the sessionizer produces the right `(user_pseudo_id, ga_session_id)` rows and the right `channel_group`. This is where Simpson's-paradox-relevant logic is locked down.
- **Reconciliation tests** are the verify-then-trust backbone: `sum(revenue)` from `fct_orders` must equal `sum(purchase_revenue_in_usd)` from raw within a tolerance of zero. Any drift fails the build and blocks the agent run, directly supporting the **100% governed SQL / 0 hallucinated metrics** target.

15.5 Data freshness and SLA

The public dataset is static, so freshness is enforced against the source's max shard. The production SLA: the latest available `events_YYYYMMDD` must be reflected in marts within the run window, and an autonomous diagnosis must complete in **under 5 minutes per run**. dbt source freshness:

```
freshness:
  warn_after: {count: 36, period: hour}
  error_after: {count: 48, period: hour}
  loaded_at_field: parse_date('%Y%m%d', _table_suffix_or_event_date)
```

A freshness error blocks the agent pipeline rather than letting it diagnose stale data.

15.6 CI/CD for analytics

GitHub Actions runs `dbt build` plus the eval harness on every PR:

```
on: pull_request
jobs:
  dbt-ci:
    steps:
      - run: dbt deps
      - run: dbt build --select state:modified+ --defer --state ./prod-manifest
      - run: dbt test --select state:modified+
      - run: python eval/run_benchmark.py # root-cause accuracy gate ≥85%
```

- **Slim CI** via `state:modified+ + --defer` builds only changed models and their children against a prod manifest, keeping byte cost under the run budget.
- The **eval harness** runs the labeled diagnosis benchmark; a PR that drops root-cause accuracy below 85% (vs the $\leq 45\%$ naive baseline) fails CI. This makes correctness a merge gate, not an afterthought.

15.7 Environments

Two physical BigQuery datasets isolate work: `helios_dev` (per-developer, prefixed schemas) and `helios_prod`. The active dataset is chosen by dbt target in `profiles.yml`. CI builds into ephemeral dev schemas; merges to `main` trigger the prod build. The semantic layer is identical across environments so the agents behave consistently dev-to-prod.

15.8 SQL code review

No SQL merges without human review of the compiled output. Reviewers check: grain is declared and tested; no `SELECT *`; partition filter present; new metrics added to the semantic layer (not hardcoded in marts); reconciliation test added for any new fact. The Critic agent is the runtime analogue — it adversarially refutes findings (mix-shift confound, insufficient sample, seasonality, data quality) before they ship.

15.9 Lineage and exposures

dbt's DAG gives column- and model-level lineage from `src_ga4` through `models/semantic`. **Exposures** declare the downstream consumers — the seven agents and the Decision Brief — so `dbt ls --select +exposure:helios_decision_brief` answers "what feeds the brief?" and impact analysis (`dbt build --select +<changed_model>+exposure:*`) shows what a change touches before it ships.

16. dbt Project Structure

16.1 Project tree

```
helios/
├─ dbt_project.yml
├─ packages.yml
├─ profiles.yml          # dev/prod targets → helios_dev / helios_prod
├─ models/
│  ├─ staging/
│  │  ├─ src_ga4.yml      # source declaration for events_* shards
│  │  ├─ stg_ga4_events.sql # 1:1 typed/renamed event rows
│  │  ├─ stg_ga4_event_params.sql # unnested key/value param long table
│  │  └─ stg_ga4_schema.yml # staging tests + docs
│  ├─ intermediate/
│  │  ├─ int_ga4_sessionized.sql # session key, landing_page, channel_group
│  │  ├─ int_ga4_funnel_steps.sql # per-session funnel-step boolean flags
│  │  └─ int_ga4_schema.yml
│  └─ marts/
│     ├─ core/
│     │  ├─ fct_sessions.sql      # 1 row / session; engagement, funnel reach
│     │  ├─ fct_funnel.sql        # 1 row / session (PK session_key); boolean reached_* flags
│     │  ├─ fct_daily_funnel.sql  # daily grain funnel counts + rates
│     │  └─ dim_users.sql         # 1 row / user_pseudo_id; first-touch attrs
```

```

├── dim_items.sql           # 1 row / item_id; category hierarchy
├── dim_channels.sql       # source/medium → channel_group map
├── dim_date.sql           # date spine, week, day-of-week
├── core__schema.yml
├── finance/
│   ├── fct_orders.sql     # 1 row / transaction_id; revenue, aov inputs
│   ├── fct_order_items.sql # 1 row / transaction_id / item
│   └── finance__schema.yml
├── growth/
│   ├── fct_funnel_by_dim.sql # funnel rollup by canonical dimensions
│   ├── fct_cohorts.sql      # weekly acquisition cohorts (retention input)
│   └── growth__schema.yml
├── semantic/
│   ├── semantic_models.yml  # governed metric definitions (canonical names)
│   └── metrics__schema.yml
├── macros/
│   ├── get_event_param.sql   # extract a typed value from event_params
│   ├── channel_group.sql     # channel_group_case(): single source of truth (rules in 04-semantics 12.5)
│   ├── sessionize.sql        # build (user_pseudo_id, ga_session_id) key
│   └── test_revenue_reconciles.sql # custom generic test
├── seeds/
│   └── channel_group_mapping.csv # source/medium → channel_group seed
├── tests/
│   └── assert_session_conversion_rate_bounds.sql # singular data test
├── snapshots/
│   └── snap_dim_items.sql     # SCD2 on item price/category
├── analyses/
│   └── adhoc_mix_shift_explore.sql # non-materialized exploration
├── exposures/
│   └── exposures.yml          # agents + Decision Brief consumers

```

16.2 Model file catalog

Model	Layer	Grain	Purpose
stg_ga4__events	staging	event	typed/renamed 1:1 view of <code>events_*</code>
stg_ga4__event_params	staging	event×param	unnested <code>event_params</code> long table
int_ga4__sessionized	intermediate	session	session key, <code>landing_page</code> , <code>channel_group</code> , <code>is_new_user</code>
int_ga4__funnel_steps	intermediate	session	boolean reach flags per macro funnel step
fct_sessions	core	session	engagement, funnel reach, RPS inputs
fct_funnel	core	session (PK <code>session_key</code>)	one row per session with boolean <code>reached_*</code> step flags; step-to-step rates computed from those flags (or from <code>fct_daily_funnel</code>)
fct_daily_funnel	core	day	daily <code>sessions</code> , step sessions, rates
dim_users	core	<code>user_pseudo_id</code>	first-touch attrs, new/returning
dim_items	core	<code>item_id</code>	item/category hierarchy
dim_channels	core	<code>channel_group</code>	conformed channel dim

Model	Layer	Grain	Purpose
dim_date	core	day	date spine
fct_orders	finance	transaction_id	revenue , gross_revenue , net_revenue , aov inputs
fct_order_items	finance	transaction_id×item	items_per_transaction , item revenue
fct_funnel_by_dim	growth	day×dimension	decomposition input (mix vs rate)
fct_cohorts	growth	cohort_week×age	retention input for stats- mcp.cohort_retention

16.3 sources.yml (src_ga4)

```

version: 2
sources:
  - name: src_ga4
    database: bigquery-public-data
    schema: ga4_obfuscated_sample_ecommerce
    loaded_at_field: parse_date('%Y%m%d', event_date)
    freshness:
      warn_after: {count: 36, period: hour}
      error_after: {count: 48, period: hour}
    tables:
      - name: events
        identifier: "events_*" # date-sharded wildcard
        description: "GA4 export; rows are events; events_YYYYMMDD shards"
        columns:
          - name: event_date
            tests: [not_null]
          - name: user_pseudo_id
            description: "de facto user key (user_id almost always null)"

```

16.4 schema.yml with tests

```

version: 2
models:
  - name: fct_orders
    description: "One row per transaction_id with revenue measures."
    columns:
      - name: transaction_id
        tests: [unique, not_null]
      - name: revenue
        tests:
          - not_null
          - dbt_utils.accepted_range: {min_value: 0}
      - name: channel_group
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
    tests:
      - revenue_reconciles: # custom generic test
        column_name: revenue
        tolerance: 0
  - name: dim_channels

```

```

columns:
  - name: channel_group
  tests:
    - not_null
    - accepted_values:
        values: ['Direct','Organic Search','Paid Search','Display',
                 'Paid Social','Organic Social','Email','Affiliates',
                 'Referral','Other']

```

16.5 Custom generic test (test_revenue_reconciles.sql)

```

{% test revenue_reconciles(model, column_name, tolerance=0) %}
with mart as (select sum({{ column_name }}) as v from {{ model }}),
raw as (
  select sum(ecommerce.purchase_revenue_in_usd) as v
  from {{ source('src_ga4','events') }}
  where event_name = 'purchase'
)
select mart.v as mart_v, raw.v as raw_v
from mart cross join raw
where abs(coalesce(mart.v,0) - coalesce(raw.v,0)) > {{ tolerance }}
{% endtest %}

```

16.6 Key macros

```

-- get_event_param.sql : typed extraction from the event_params ARRAY
{% macro get_event_param(key, type='string') %}
(select ep.value.{{ type }}_value
 from unnest(event_params) ep
 where ep.key = '{{ key }}' limit 1)
{% endmacro %}

-- channel_group.sql : SINGLE SOURCE OF TRUTH for channel grouping.
-- This macro is the only place the classification logic lives; the rules it
-- encodes are documented authoritatively in 04-semantics section 12.5.
-- Body is an exact copy of the channel_group_case CASE SQL from 12.5
-- (10 groups only, has_gclid-based Paid detection, top-to-bottom precedence,
-- no "Paid Other" branch).
{% macro channel_group_case() %}
CASE
  -- Direct
  WHEN LOWER(session_medium) IN ('(none)','(not set)','') AND LOWER(session_source) IN ('(direct)','')
  THEN 'Direct'
  -- Paid Search
  WHEN (REGEXP_CONTAINS(LOWER(session_medium), r'^(cpc|ppc|paid|paidsearch)$') OR has_gclid)
  AND REGEXP_CONTAINS(LOWER(session_source), r'google|bing|yahoo|duckduckgo|ecosia|baidu|yandex')
  THEN 'Paid Search'
  -- Paid Social
  WHEN REGEXP_CONTAINS(LOWER(session_medium), r'^(cpc|ppc|paid.*)$')
  AND REGEXP_CONTAINS(LOWER(session_source),
    r'facebook|instagram|fb|twitter|x\.com|tiktok|linkedin|pinterest|reddit|snapchat|youtube')
  THEN 'Paid Social'
  -- Display
  WHEN REGEXP_CONTAINS(LOWER(session_medium), r'^(display|banner|expandable|interstitial|cpm)$')
  THEN 'Display'
  -- Organic Search
  WHEN LOWER(session_medium) = 'organic'
  OR REGEXP_CONTAINS(LOWER(session_source), r'google|bing|yahoo|duckduckgo|ecosia|baidu|yandex')

```

```

    THEN 'Organic Search'
-- Organic Social
WHEN LOWER(session_medium) IN ('social','social-network','social-media','sm','social network','social media')
OR REGEXP_CONTAINS(LOWER(session_source),
    r'facebook|instagram|fb|twitter|tiktok|linkedin|pinterest|reddit|snapchat|youtube')
    THEN 'Organic Social'
-- Email
WHEN LOWER(session_medium) IN ('email','e-mail','e_mail','newsletter')
OR REGEXP_CONTAINS(LOWER(session_source), r'email|newsletter')
    THEN 'Email'
-- Affiliates
WHEN LOWER(session_medium) IN ('affiliate','affiliates')
    THEN 'Affiliates'
-- Referral
WHEN LOWER(session_medium) IN ('referral','link')
    THEN 'Referral'
ELSE 'Other'
END
{% endmacro %}

-- sessionize.sql : build the session key (honors the traffic_source gotcha)
{% macro sessionize() %}
concat(user_pseudo_id, '-',
    cast({% get_event_param('ga_session_id','int') %} as string))
{% endmacro %}

```

16.7 dbt_project.yml

```

name: helios
version: '1.0.0'
profile: helios
require-dbt-version: "≥1.7.0"
models:
  helios:
    staging:      {+materialized: view}
    intermediate: {+materialized: ephemeral}
    marts:
      core:
        +materialized: incremental
        +incremental_strategy: insert_overwrite
        +partition_by: {field: event_date, data_type: date, granularity: day}
        +cluster_by: ['device_category','channel_group']
      finance: {+materialized: table}
      growth: {+materialized: table}
      semantic: {+materialized: view}
seeds:
  helios: {channel_group_mapping: {+column_types: {medium: string}}}

```

16.8 packages.yml

```

packages:
- package: dbt-labs/dbt_utils
  version: ["≥1.1.0", "<2.0.0"]

```

16.9 Sample staging model (stg_ga4__events.sql)


```

{{ config(materialized='view') }}
select
  parse_date('%Y%m%d', event_date)                as event_date,
  event_timestamp,
  event_name,
  user_pseudo_id,
  {{ sessionize() }}                               as session_key,
  {{ get_event_param('ga_session_id','int') }}     as ga_session_id,
  {{ get_event_param('page_location') }}           as page_location,
  {{ get_event_param('source') }}                 as session_source,
  {{ get_event_param('medium') }}                 as session_medium,
  device.category                                 as device_category,
  device.web_info.browser                        as browser,
  geo.country                                    as country,
  ecommerce.transaction_id,
  ecommerce.purchase_revenue_in_usd
from {{ source('src_ga4','events') }}
{% if is_incremental() %}
where parse_date('%Y%m%d', event_date)
      ≥ date_sub(_dbt_max_partition, interval 3 day)
{% endif %}

```

16.10 Sample mart model (fct_daily_funnel.sql)

```

{{ config(materialized='incremental', incremental_strategy='insert_overwrite',
  partition_by={'field':'event_date','data_type':'date','granularity':'day'}) }}
select
  event_date,
  count(distinct session_key)                as sessions,
  count(distinct if(reached_view_item, session_key, null)) as view_item_sessions,
  count(distinct if(reached_add_to_cart, session_key, null)) as add_to_cart_sessions,
  count(distinct if(reached_begin_checkout, session_key, null)) as begin_checkout_sessions,
  count(distinct if(reached_add_shipping_info, session_key, null)) as add_shipping_info_sessions,
  count(distinct if(reached_add_payment_info, session_key, null)) as add_payment_info_sessions,
  count(distinct if(reached_purchase, session_key, null)) as purchasing_sessions,
  safe_divide(count(distinct if(reached_purchase, session_key, null)),
    count(distinct session_key))
    as session_conversion_rate
from {{ ref('int_ga4__funnel_steps') }}
group by event_date

```

17. BigQuery Architecture

17.1 Dataset layout

Dataset	Contents	Materialization	Access
bigquery-public-data.ga4_obfuscated_sample_ecommerce	raw events_* shards	external (read-only)	roles/bigquery.dataViewer
helios_staging	stg_ga4__*	views	service account RW
helios_marts	fct_*, dim_* (core/finance/growth)	incremental tables	service account RW

Dataset	Contents	Materialization	Access
helios_semantic	governed metric views	views	semantic-mcp RO
helios_eval	labeled benchmark, snapshots of agent outputs	tables	eval harness RW

Separating `semantic` from `marts` enforces the grounding principle: `semantic-mcp` is granted read on `helios_semantic` only, so the LLM physically cannot bypass governance to hit raw facts.

17.2 Partitioning and clustering

- **Partition by `event_date (DATE)`.** Every fact and staging model is day-partitioned. The diagnosis pipeline always scopes a window ($t_0 \rightarrow t_1$ for `decompose_change`, a series for `detect_anomaly`), so date partitioning gives near-perfect pruning: a 14-day diagnosis scans 14 of ~90 partitions.
- **Cluster by `device_category, channel_group`.** These are the two highest-cardinality, most-queried canonical dimensions (the mix-shift decomposition almost always slices by them). Clustering co-locates rows so block-level filtering drops bytes scanned again after partition pruning.

Rationale: partition pruning is coarse (whole days); clustering is the fine-grained second filter on the dimensions the Decompose agent uses to separate **mix effect** from **rate effect**.

17.3 Materialization choices

Model class	Materialization	Why
staging	view	thin rename/cast; no storage cost; always fresh
intermediate	ephemeral	inlined as CTEs; no object proliferation
core facts	incremental (<code>insert_overwrite</code>)	day-grain, idempotent partition replacement
finance/growth facts	table	small, fully rebuilt cheaply
semantic	view	governance layer; must reflect marts instantly
daily rollups	materialized view (optional)	auto-maintained pre-aggregates for hot dashboards

`insert_overwrite` is chosen over `merge` because partition-atomic overwrite is cheaper and inherently idempotent for the date-sharded GA4 model — no surrogate-key dedup needed.

17.4 Cost controls

The system runs autonomously and must stay **under a fixed BigQuery byte budget per run**. Controls, layered:

1. **Partition pruning** — every query filters `event_date` BETWEEN the diagnosis window. `_dbt_max_partition` drives incremental lookback.
2. **`require_partition_filter = true`** on all partitioned marts, so any query missing a date predicate errors instead of full-scanning.

3. **maximum_bytes_billed** set on the connection (e.g. per-query cap), so a runaway query is killed by BigQuery, not discovered on the bill.
4. **Dry-run budgets** — `warehouse-mcp.dry_run(sql)` returns estimated bytes/cost before any execution; a per-query cap and a per-run cumulative cap are enforced (see 17.8).
5. **Shard pruning** — when reading raw `events_*`, filter `_TABLE_SUFFIX BETWEEN '20210101' AND '20210114'` so only relevant daily tables are read.
6. **No `SELECT *`** — column projection is mandatory (GA4 rows are wide with nested ARRAYS; `SELECT *` materializes megabytes of unused structs).

17.5 On-demand vs slot pricing

Helios defaults to **on-demand** (pay per byte scanned), which aligns with the byte-budget guardrail and the bursty, scheduled-run workload — there are no idle slots to amortize. If run frequency rises to continuous monitoring, a small **flat-rate / autoscaling slot reservation** caps spend and removes per-byte variance; the dry-run guardrail still governs bytes for query hygiene regardless of pricing model. The decision pivots on duty cycle: on-demand below it, reserved slots above the breakeven where per-byte cost exceeds slot rent.

17.6 IAM, service accounts, least privilege

- **sa-helios-runner** — runs dbt + agents in prod. `roles/bigquery.dataEditor` on `helios_*` datasets; `roles/bigquery.dataViewer` on the public source; `roles/bigquery.jobUser` to run queries. No project-level admin.
- **sa-helios-semantic** — backs `semantic-mcp`. **Read-only** on `helios_semantic` only. This is the technical enforcement of "the LLM never authors raw SQL": the credential it runs under cannot read raw or even mart tables directly.
- **sa-helios-ci** — GitHub Actions. RW on ephemeral CI schemas, RO on prod manifest. Scoped via Workload Identity Federation (no long-lived keys).
- Developers get RW only on their personal `helios_dev` schema.

17.7 GA4 query-optimization patterns

```
-- Prune shards + project only needed columns; never SELECT *
select event_timestamp, user_pseudo_id, event_name,
       (select value.int_value from unnest(event_params)
        where key = 'ga_session_id') as ga_session_id
from `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*`
where _TABLE_SUFFIX between '20210101' and '20210114' -- shard prune
      and event_name in ('session_start','purchase')    -- predicate pushdown
```

- Unnest `event_params` once in staging (`stg_ga4__event_params`); never repeat correlated UNNEST subqueries downstream.
- Filter `_TABLE_SUFFIX` on the raw wildcard; filter `event_date` on partitioned marts.
- Honor the **traffic_source gotcha**: prefer session-scoped `event_params.source/medium` for `channel_group`; fall back to user first-touch `traffic_source` only when session scope is null.

17.8 How the dry_run guardrail enforces the byte budget

`warehouse-mcp.dry_run(sql)` performs a BigQuery dry run (`use_query_cache=False` , `dry_run=True`) and returns `total_bytes_processed` without executing. The guardrail wraps every execution path:

```
def guarded_run(sql, per_query_cap, run_state):
    est = warehouse_mcp.dry_run(sql)          # bytes, no execution, no cost
    if est.bytes > per_query_cap:
        raise BudgetError("query exceeds per-query cap")
    if run_state.bytes_used + est.bytes > run_state.run_cap:
        raise BudgetError("run byte budget exhausted")
    rows = warehouse_mcp.run_query(sql)       # only now does it cost
    run_state.bytes_used += est.bytes
    return rows
```

Every SQL the agents execute — composed exclusively through `semantic-mcp.build_query` — passes the dry run first. This keeps **query cost per run under the fixed budget**, contributes to **time-to-diagnosis under 5 minutes**, and, combined with reconciliation and the Critic, upholds **0 hallucinated columns/metrics and 100% governed SQL**.